

第二届“良文杯” 统计建模与AI预测挑战赛 (金融大数据主题)

平台说明会&模型训练讲座

时 间：2026年4月1日



目录

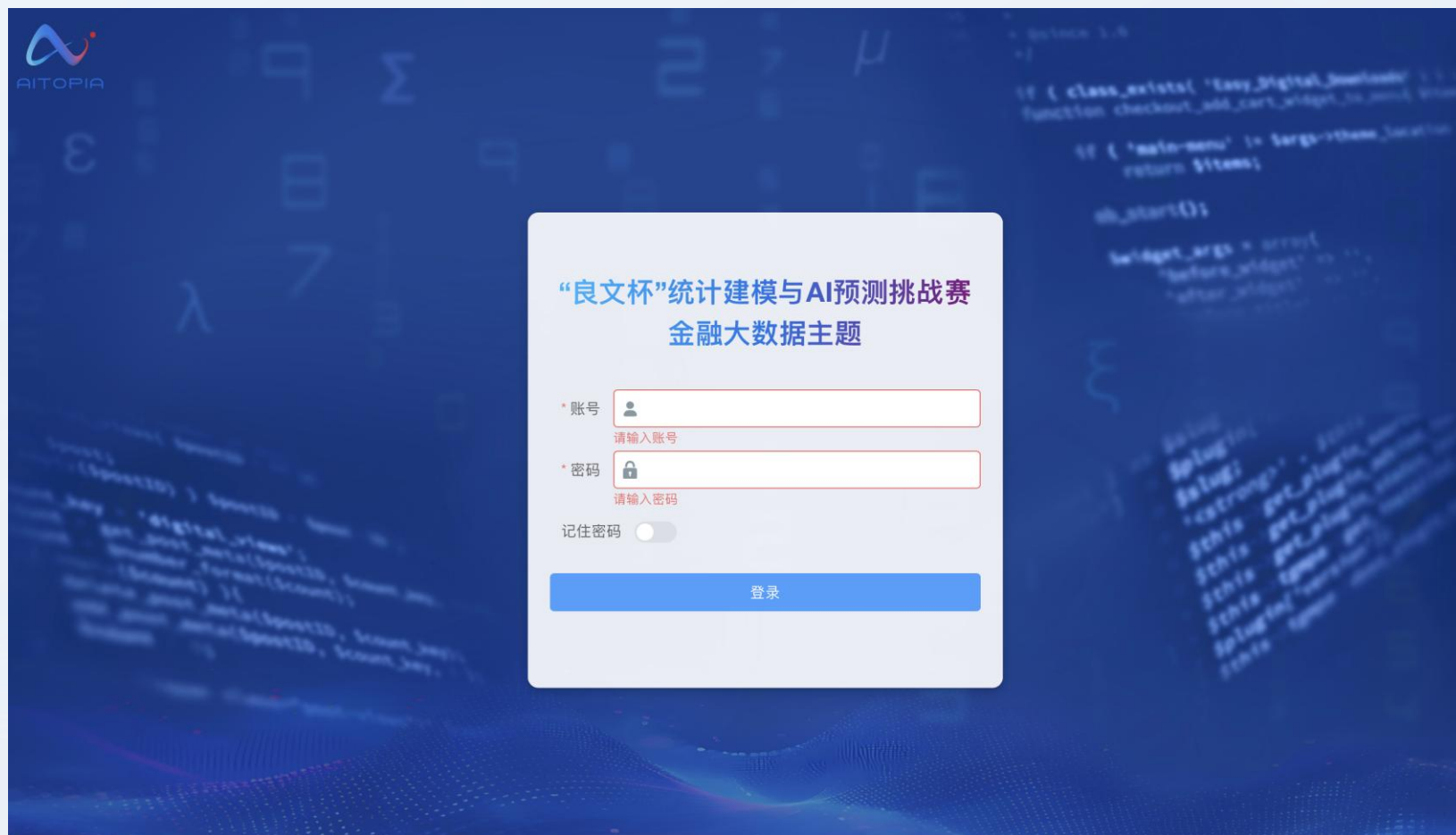
CONTENTS

- 1. 平台使用和评测规则
- 2. 提交准备和测试说明
- 3. 模型训练初步尝试与优化

1.1 平台登录

- 平台链接: <https://fin-bdc.aitopia.cn/>
- 账号密码: 群公告报名查询链接签收

1. 平台账号: 三位数唯一随机码
2. 平台密码: 初始随机密码【建议修改】
3. 队名(平台显示): 平台账号-队名



队名	良文杯
平台账号	999
平台密码	123123123
队名(平台显示)	999-良文杯
队长姓名	吴茜
队长学校	清华大学
队长联系方式	xx

1.2 平台功能：指南和评测

首次登录
请修改密码

指南

- 竞赛规则
- 数据介绍
- 评测指南

榜单

- 公榜评测
- 私榜评测

训练&验证：本地

测试：评测平台

根据测试结果迭代

"良文杯"统计建模与AI预测挑战赛 (金融大数据主题)

修改密码
退出登录

竞赛规则

预测目标：利用股票过往及当前数据预测未来中间价的移动方向

输入数据：

- 行情频率：3秒一个数据点（也称为1个tick的snapshot）
- 每个数据点包括当前最新成交价/过去3秒内的成交金额/五档量价等行情数据
- 训练集中每个数据点包含5个预测标签的标注

允许利用过去100tick（包含当前tick）的数据，预测未来n个tick后的中间价移动方向（n取5, 10, 20, 40, 60）

中间价midprice

预测目标：利用股票过往及当前数据预测未来中间价的移动方向

中间价定义：

- 当 ask1 和 bid1 都不为零时，midprice 等于 $(ask1 + bid1) / 2$ ，即卖一价和买一价的平均值
- 当 bid1 为零时（例如跌停），midprice 等于 ask1
- 当 ask1 为零时（例如涨停），midprice 等于 bid1

预测时间跨度：5、10、20、40、60个tick，5个预测任务

- 即在t时刻，分别预测t+5tick, t+10tick, t+20tick, t+40tick, t+60tick以后
- 最新中间价相较于t时刻的中间价：下跌/不变/上涨
- 取5个预测任务中的最高得分参与排行

1.3 平台提交&公榜评测

- 平台提供CPU评测模式和GPU评测模式，用户可以根据模型需求选择评测模式
- 同一账号**每12小时**仅限提交1次评测任务，建议合理规划本地模型优化与平台评测节奏
 - 如果任务异常，请下载日志查看错误信息并修改bug，删除异常任务后可重新提交
 - 如提交后12小时后仍未有结果，可QQ群联系AITOPIA管理员
- 单次模型评测时长*限制在**3小时**内，超时任务将自动终止并报异常
- 模型文件不超过**2GB**，FP32精度

“良文杯”统计建模与AI预测挑战赛（金融大数据主题）							测试用户 ▾
- 竞赛规则 - 数据介绍 - 评测指南 公榜评测 - 私榜评测	评测任务		公榜成绩				
	请输入队名		创建评测任务		开放提交时间段 2025-04-16 18:00:00 至 2025-05-26 00:00:00		
	任务ID	队名	任务创建时间	评测模式	任务评测时长	任务状态	操作
	68	aitopia	2025-04-16 17:36:19	GPU评测	00小时59分26秒	评测完成	删除 下载

任务状态

- 任务创建成功
- 等待调度/调度成功
- 开始创建环境/环境创建成功
- 进入评测队列/评测中/评测完成
- 发生异常，请下载日志查看错误信息

任务评测时长

- 从任务进入“评测中”到“评测完成”计时
- 不包括评测环境创建时间

1.4 公榜成绩查看

- “预测任务”可筛选，其他字段可排序
- “模型评分”为公榜排行依据
- 竞赛规则-平台的评价标准

“良文杯”统计建模与AI预测挑战赛（金融大数据主题）

测试用户 ▾

竞赛规则

数据介绍

评测指南

公榜评测

私榜评测

评测任务

公榜成绩

任务ID	队名	准确率	召回率	F0.5分数	收益率	单次收益率	模型评分
68	aitopia	0.329426	0.294724	0.321846	8.78137	4.12517e-5	-4.14217e-8
66	www	0.329412	0.294791	0.321851	8.74549		639e-8
65	ttt	0.263844	0.0373248	0.119182	0.563608		061e-8
65	ttt	0.236661	0.0376285	0.115001	0.702859	2.08811e-5	-1.65292e-8

往届测试数据

预测任务 ▾

- ☐ label_5
- ☐ label_10
- ☐ label_20
- ☐ label_40
- ☐ label_60

筛选 重置

1.5模型评分： 累计收益率（扣除手续费）

■ 收益率（扣除手续费）计算：

- 预测上涨（label=2）视为：此时出手买入，到预测的5/10/20/40/60tick后卖出，买卖双边均收取手续费0.01%
- 预测不变（label=1）视为：不出手
- 预测下跌（label=0）视为：此时卖出，到预测的5/10/20/40/60tick后买入（假设有底仓），买卖双边均收取手续费0.01%

$$\text{pnl}_{\text{single}} = \frac{(\text{label} - 1)(\text{midprice}_{t+n} - \text{midprice}_t) - 0.0001 \cdot |\text{label} - 1| \cdot [(\text{midprice}_{t+n} + 1) + (\text{midprice}_t + 1)]}{\text{midprice}_t + 1}$$

- 累计收益率：所有预测上涨/下跌的收益率之和
- 单次收益率：累计收益率 / （预测的上涨总数+预测的下跌总数）
 - 特别注意是预测为上涨或下跌的总数

■ 模型评分： 累计收益率（扣除手续费）

真实市场环境：手续费

真实评测标准：收益率

简化：不考虑交易执行

补充：股票买卖手续费

- 本次赛题，仅考虑佣金，不考虑印花税（实际非常重要）

股票交易的手续费

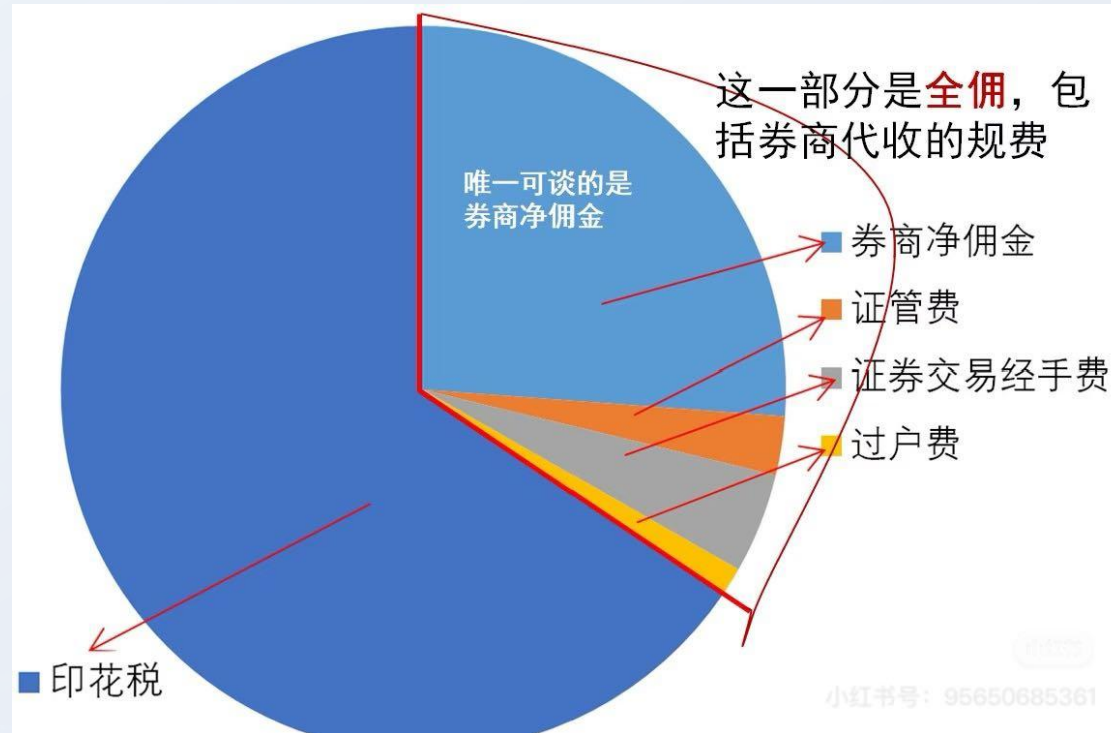
费用类别	具体项目	收取方	收费标准	收取方向
券商佣金	券商净佣金	证券公司	万0.3~2.5 单笔最低5元	买入、卖出 双向收取
交易所 规费	证管费	证监会	万0.2	买入、卖出 双向收取
	证券交易经手费	证券交易所	万0.341	
	过户费	中国结算公司	万0.1	
国家税收	印花税	国家税务机关	万5	仅在卖出时收取

唯一可谈部分是：
券商净佣金

费率固定，万0.641
 $=0.2+0.341+0.1$ 。
由券商代收后上缴。

全佣

小红书号：95650685361



- 出手阈值和次数？
- 准确率 vs 召回率 (F0.5)
- 5个任务：标签？预测难度？收益率？

1.6 私榜评测

- 平台将自动选取用户在公榜评测中最后一次成功提交且结果有效的模型用于私榜评测
 - 请确保在开放提交时间段内，在公榜评测中最近创建的任务中的模型，即为参与私榜评测的最终版本
 - 可以通过删除任务，调整最终参与私榜评测的模型
- 平台调用模型时间：公榜评测结束1-2天后
- 私榜评价标准：公榜一致
- 决赛入选标准：私榜排名，取5个任务中表现最好的参与排名

“良文杯”统计建模与AI预测挑战赛（金融大数据主题）

测试用户 ▾

竞赛规则

数据介绍

评测指南

公榜评测

私榜评测

评测任务

私榜成绩

预测任务 label_5 ▾

成绩导出

排名	队名	任务ID	准确率 ▾	召回率 ▾	F0.5分数 ▾	收益率 ▾	单次收益率 ▾	模型评分 ▾



目录

CONTENTS

- 1. 平台使用说明
- 2. 提交准备和测试说明
- 3. 模型训练初步尝试与优化

2.2 提交包准备

Step0: 必要文件

■ 为正确评测模型，需按引导文字准备以下提交文件：

■ 1. 模型定义及权重文件：

■ model.py

■ model.pth

■ 2. 提交说明：config.json

■ python_version: 你用到了哪个python版本？

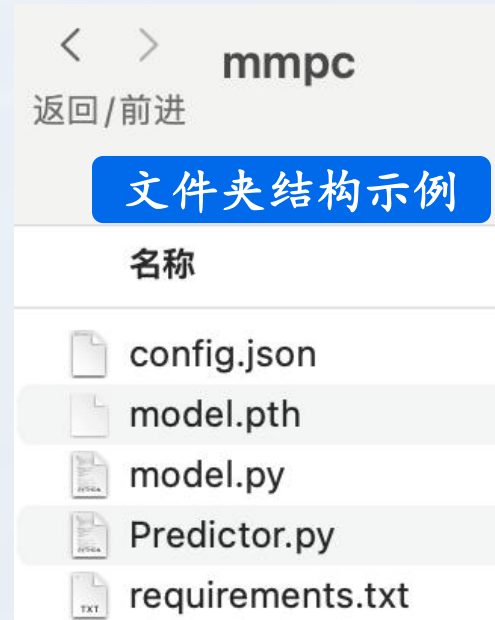
■ batch: 你每次预测多少数据量？

■ feature: 你用到了哪些数据列？

■ label: 本次提交需要测试哪个label？

■ 3. 环境依赖说明：requirements.txt

■ 4. 根据模板写自己的预测类：Predictor.py



文件夹名称可自由选择：mmpc
(midprice move prediction challenge)

注意

- 平台会把压缩包中文件提取到同一目录
- 为了防止python导入时路径出错，请将所有文件直接存放在同一文件夹内，不要创建子文件夹

2.2 提交包准备

Step1: 准备模型定义及权重

■ 假如有了一个训练完的模型：

- 将训练好的模型参数文件（例如model.pth）复制到提交文件夹（如mmpc）目录下
- 将模型定义抽离成文件（例如model.py），复制到提交文件夹（如mmpc）目录下

2.2 提交包准备

Step2: 通知测试平台 config.json

- 指定评测代码依赖的python版本

衍生特征、特征较多

提升数据处理效率

- 为了给同学们提供最大的自由度，平台允许在每次提交时，通过config.json 决定 “我要使用哪些数据，测试什么，每批次测试多少数据量”

```
{
  "python_version": "3.11",
  "batch": 1024,
  "feature": [
    "volume_delta",
    "amount_delta",
    "bid1", "bsize1",
    "bid2", "bsize2",
    "bid3", "bsize3",
    "bid4", "bsize4",
    "bid5", "bsize5",
    "ask1", "asize1",
    "ask2", "asize2",
    "ask3", "asize3",
    "ask4", "asize4",
    "ask5", "asize5"
  ],
  "label": ["label_5"]
}
```

```
{
  "python_version": "3.11",
  "batch": 1024,
  "feature": ["open", "high", "low", "close", "volume_delta", "amount_delta", "bid1", "bsize1", "bid2", "bsize2", "bid3", "bsize3", "bid4", "bsize4", "bid5", "bsize5", "ask1", "asize1", "ask2", "asize2", "ask3", "asize3", "ask4", "asize4", "ask5", "asize5"],
  "label": ["label_5", "label_10", "label_20", "label_40", "label_60"]
}
```

- 我只需要5档行情，只预测5tick后的涨跌幅
- 每批次数据量 1024* pd.DataFrame(100, 22)

- 我需要全部数据，同时预测所有label的涨跌幅
- 每批次数据量 1024* pd.DataFrame(100, 154)

2.2 提交包准备

Step3: 确定有哪些依赖包

- 为了平台能够自动安装对应环境，需要同学们导出requirement.txt
 - 用于同学们在训练中，往往会用到不同版本的python package包，例如有的同学使用pytorch 1.4，有的使用pytorch 2.0.
- 建议新建推理环境，使用pipreqs包导出，命令示例如下：

```
# 创建文件夹
mkdir file_folder
# 进入文件夹
cd file_folder
# 将模型文件、Predictor.py 复制到文件夹
cp -r /path/to/model.pkl /path/to/Predictor.py .
# 生成依赖文件
pipreqs ./ --encoding=utf8 --force
# 验证依赖文件
conda create -n test python=3.10 (指定python版本)
conda activate test
pip install -r requirements.txt
```

2.2 提交包准备

Step4: 创建Predictor.py

根据你的模型和预处理，创建/修改Predictor.py

- 关键位置1: 导入模型定义
 - pytorch的模型保存分为保存整个模型和只保存权重
 - 建议只保存权重
 - 在Predictor.py中import模型定义
 - 采用**相对位置**，from .model import *

保存整个模型:

```
torch.save(model, 'save.pt')
```

只保存训练好的权重:

```
torch.save(model.state_dict(), 'save.pt')
```

```
import os
import numpy as np
from .model import *
```

2.2 提交包准备

Step4: 创建Predictor.py

根据你的模型和预处理，创建/修改Predictor.py

■ 关键位置2: `__init__`

- 需要确保模型权重能被正常加载
- 为了减少路径错误，采用**指定路径**

```
class Predictor():
    Tabnine | Edit | Test | Explain | Document
    def __init__(self):
        # 指定模型路径，不使用相对路径
        pth_path = os.path.join(os.path.dirname(__file__), 'model.pth')
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        # 加载模型并移动到对应设备，假设模型是整个模型保存，如果是参数字典需要初始化结构
        self.model = self.load_model(pth_path)
        self.model.to(self.device)
        self.model.eval() # 切换到评估模式以关闭 dropout、batchnorm 等层
```


2.2 提交包准备

Step4: 创建Predictor.py

根据你的模型和预处理，创建/修改Predictor.py

■ 关键位置3: predict()方法

■ 单次会输入形如List[pd.DataFrame]的数据:

- List的大小为batch，即配置文件中的batch
- pd.DataFrame中数据为过去100个tick的数据
- pd.DataFrame的列名为配置文件中的feature

■ 函数应返回List[List[int]]格式的预测结果

- 外层List的大小为batch，即配置文件中的batch
- 内层List的大小为label的个数，即配置文件中的label的个数
- int值为预测标签（标签为0/1/2）

```
Tabnine | Edit | Test | Explain | Document
def predict(self, x: List[pd.DataFrame]) -> List[List[int]]:
    # 对输入数据进行预处理
    x_hat = self.preprocess(x)
    with torch.no_grad(): # 禁用梯度计算，加速推理
        y = []
        for i in range(5):
            y_pred = self.model(x_hat).cpu().numpy()
            y.append(np.argmax(y_pred,axis=1).tolist())
        y = np.array(y).T.tolist()
    # 确保返回格式为 List[List[int]]
    if isinstance(y[0], list):
        return y
    else:
        return [y]
```

2.3 验证&打包

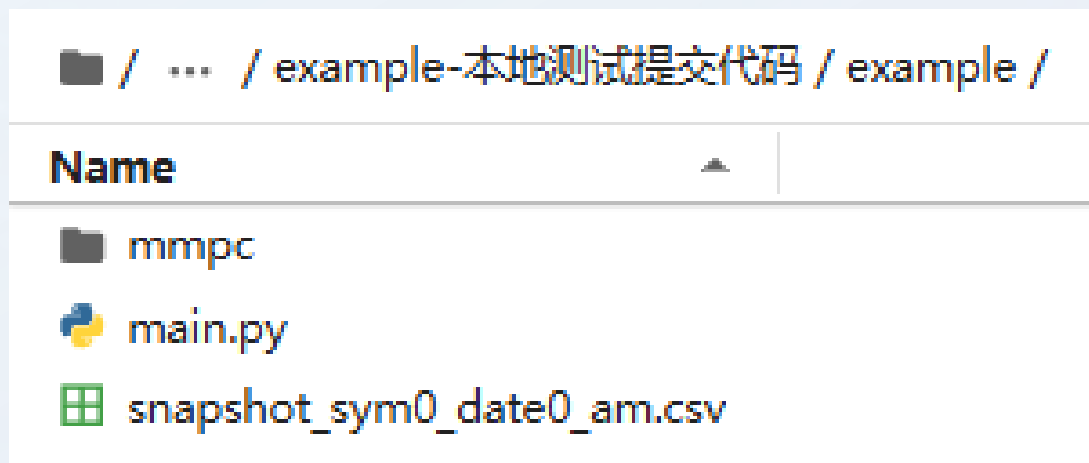
- 如果推理环境中程序运行没有问题，就可以打包了
- 将提交文件夹（如mmpc）打包成zip文件，上传平台公榜评测

回顾：推理环境创建

```
# 创建文件夹
mkdir file_folder
# 进入文件夹
cd file_folder
# 将模型文件、Predictor.py 复制到文件夹
cp -r /path/to/model.pkl /path/to/Predictor.py .
# 生成依赖文件
pipreqs ./ --encoding=utf8 --force
# 验证依赖文件
conda create -n test python=3.10 (指定python版本)
conda activate test
pip install -r requirements.txt
```

2.3 验证&打包

- 根据往年经验，比赛还提供了本地测试代码example
 - 可以在本地测试代码在平台上是否能成功运行
 - 提高验证效率



2.4 测试方法

- 测试方式为交互式调用
 - 参赛者需要提交源代码及模型
- 代码需提供一个Predictor类，对batch组数据点进行一次预测
 - 类应至少具有_init_方法和predict方法：
 - `def predict(x: List[pd.DataFrame]) -> List[List[int]]:` 确保输入输出形状的一致性
 - 评测程序会多次调用predict，得到预测并汇总计算P&L
- 为了公平性与数据保密性，评测程序会打乱测试点输入顺序
 - 不同次输入的数据无先后关系
 - date: 日期会置为0，无意义
 - sym: 可能的范围0-4（可能有不来自于5只训练集股票的数据）
 - time: time会保留实际交易时间，以备参赛模型会根据时段不同做推理



目录

CONTENTS

- 1. 平台使用说明
- 2. 提交准备和测试说明
- 3. 模型训练初步尝试与优化

Step1：检查并分析数据（此处展示此部分字段）

字段	含义	说明
date	日期	去掉实际日期，但保留序号，取值范围0~119，保留跨标的的可比性
time	时间戳	保留实际时间戳，3s一档行情
sym	股票标的	仅序号，5只股票分别为0~4
close	最新成交价	无量纲，以相对昨收的涨跌幅表示，-1~1，即跌100%到涨100%（实际取值范围更小），下同
open	开盘价	
high	当前最高价	
low	当前最低价	
amount_delta	成交金额变化	从上个tick到当前tick发生的成交金额，单位：元
volume_delta	成交量变化	从上个tick到当前tick发生的成交量，无量纲，以换手率%表示
bid1 ~ bid10	买一价-买十价	无量纲，以涨跌幅表示
ask1 ~ ask10	卖一价-卖十价	无量纲，以涨跌幅表示
bsize1 ~ bsize10	买一量-买十量	无量纲，以换手率%表示
asize1 ~ asize10	卖一量-卖十量	无量纲，以换手率%表示
midprice	中间价	无量纲，以涨跌幅表示
label_5, label_10, label_20, label_40, label_60	5tick, 10tick, 20tick, 40tick, 60tick后价格移动方向	当前tick中间价相对于5tick, 10tick, 20tick, 40tick, 60tick之前的移动方向，0为下跌，1为不变，2为上涨

上节提问：n_close的实际取值范围？
-20%~20%

0.0001表示订单量归一化为换手率0.0001%

5个预测任务

Step1 : 检查并分析数据

- 是否有缺失值? 可能挺多的? 缺20%以上可以直接去掉
- 是否有异常值? 变化幅度, 查看分布
- 是否有重复值? 价格可能不变, amount不太可能
- 样本是否均衡? 涨、跌、不变的分布是否均衡
- 是否需要抽样?
- 变量是否需要转换?
- 是否需要增加新的特征?

Step2：初步尝试——以SVM为例进行分类预测

■ 首先导入数据

- 一个连续交易时段只有大约2000个点（9:40-11:20，13:10-14:50）
- 将多个交易时段拼接在一起

2. 载入多个交易日的数据

```
file_dir = "./2026liangwen/2026train_set"

sym = 4
dates = list(range(12))

# 收集所有DataFrame到列表
df_list = []
for date in dates:
    if (date & 1):
        file_name = f"snapshot_sym{sym}_date{date//2}_am.parquet"
    else:
        file_name = f"snapshot_sym{sym}_date{date//2}_pm.parquet"
    df_list.append(pd.read_parquet(os.path.join(file_dir, file_name)))

# 最后一次性合并（性能更好）
df = pd.concat(df_list, ignore_index=True)
```

注意代码性能

Step2 : 初步尝试——以SVM为例进行分类预测

- 选择想要输入给模型的特征和标注数据
- 选择划分训练集和测试集
- 时刻注意检查数据形状！

df									
	date	sym	time	open	high	low	close	volume_delta	amount_delta
0	0	4	13:10:00	0.016619	0.020057	-0.014212	-0.011576	0.000053	25872.0
1	0	4	13:10:03	0.016619	0.020057	-0.014212	-0.011576	0.000407	198414.0
2	0	4	13:10:06	0.016619	0.020057	-0.014212	-0.011920	0.000849	413843.0
3	0	4	13:10:09	0.016619	0.020057	-0.014212	-0.011691	0.000265	129328.0
4	0	4	13:10:12	0.016619	0.020057	-0.014212	-0.011691	0.000088	43115.0
...	...	...	...	...	...	...	...	...	...
24007	5	4	11:19:48	-0.015194	-0.013362	-0.061638	-0.059375	0.000000	0.0
24008	5	4	11:19:51	-0.015194	-0.013362	-0.061638	-0.057866	0.000389	192283.0
24009	5	4	11:19:54	-0.015194	-0.013362	-0.061638	-0.058836	0.000106	52404.0
24010	5	4	11:19:57	-0.015194	-0.013362	-0.061638	-0.058836	0.000000	0.0
24011	5	4	11:20:00	-0.015194	-0.013362	-0.061638	-0.058621	0.000142	69886.0

24012 rows x 163 columns

```
# 确定有无na值, 若有要进行处理 (是否一定能用0填充)
df.isnull().values.any()
```

False

```
feature_col_names = ['bid1','bid2','bid3','bid4','bid5',\
                     'ask1','ask2','ask3','ask4','ask5']
label_col_name = ['label_5']
```

```
train_sample_nums = 20000
```

```
# 别忘了数据形状和存储连续性
```

```
train_data = np.ascontiguousarray(df[feature_col_names][:train_sample_nums].values)
train_label = df[label_col_name][:train_sample_nums].values.reshape(-1)
```

```
test_data = np.ascontiguousarray(df[feature_col_names][train_sample_nums:].values)
test_label = df[label_col_name][train_sample_nums:].values.reshape(-1)
```

```
train_data.flags
```

```
C_CONTIGUOUS : True
F_CONTIGUOUS : False
OWNDATA : True
WRITEABLE : True
ALIGNED : True
WRITEBACKIFCOPY : False
```

Step2：初步尝试——以SVM为例进行分类预测

- 做数据分析一定要了解数据！

别忘了看一下标签分布：

```
print("在训练集中：")
print("标签为0的样本个数：", sum(train_label == 0))
print("标签为1的样本个数：", sum(train_label == 1))
print("标签为2的样本个数：", sum(train_label == 2))

print("在测试集中：")
print("标签为0的样本个数：", sum(test_label == 0))
print("标签为1的样本个数：", sum(test_label == 1))
print("标签为2的样本个数：", sum(test_label == 2))
```

在训练集中：

标签为0的样本个数： 3302
标签为1的样本个数： 13223
标签为2的样本个数： 3475

在测试集中：

标签为0的样本个数： 563
标签为1的样本个数： 2887
标签为2的样本个数： 562

显著不均匀

Step2：初步尝试——以SVM为例进行分类预测

- “人生苦短，我用python”

导入sklearn包

建立svc模型并fit

训练好的模型做预测

做一下简单的训练：

```
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC
```

```
# 训练一个native svm分类器:
model = SVC()
model.fit(train_data, train_label)
```

▼ SVC ③ ②
SVC()

```
# 训练完了之后, 看看在训练集的准确率 (拟合的怎么样)
y_hat = model.predict(train_data)
y = train_label
```

Step2 : 初步尝试——以SVM为例进行分类预测

- 模型评价指标：如何评价模型结果好坏

$$\text{precision} = \frac{\text{true positive}}{\text{true positive} + \text{false positive}} = \frac{\text{true positive}}{\text{no.of predicted positive}}$$

$$\text{recall} = \frac{\text{true positive}}{\text{true positive} + \text{false negative}} = \frac{\text{true positive}}{\text{no.of actual positive}}$$

$$F_{\beta} = (1 + \beta^2) \times \frac{\text{precision} \times \text{recall}}{\beta^2 \times \text{precision} + \text{recall}}$$

交易侧重precision, F0.5

Step2 : 初步尝试——以SV

■ 结果不好怎么办？

■ 分析原因

■ 模型导致的？

■ 参数导致的？

■ 修改模型

■ 参数调整/搜索

别忘了看一下标签分布：

```
print("在训练集中：")
print("标签为0的样本个数：", sum(train_label == 0))
print("标签为1的样本个数：", sum(train_label == 1))
print("标签为2的样本个数：", sum(train_label == 2))

print("在测试集中：")
print("标签为0的样本个数：", sum(test_label == 0))
print("标签为1的样本个数：", sum(test_label == 1))
print("标签为2的样本个数：", sum(test_label == 2))
```

在训练集中：

标签为0的样本个数： 3302
标签为1的样本个数： 13223
标签为2的样本个数： 3475

在测试集中：

标签为0的样本个数： 563
标签为1的样本个数： 2887
标签为2的样本个数： 562

定义一个统计结果的函数：

```
def check_metric(y, y_hat):
    # 总体情况
    print("预测正确的标签数：", sum(y_hat == y))
    print("总体正确率：", sum(y_hat == y)/len(y_hat))

    # 分标签查看：
    print("真实标签为0样本的正确预测个数：", sum(y[y == 0] == y_hat[y == 0]))
    print("真实标签为1样本的正确预测个数：", sum(y[y == 1] == y_hat[y == 1]))
    print("真实标签为2样本的正确预测个数：", sum(y[y == 2] == y_hat[y == 2]))

    ## 我们更关心上涨下跌情况的预测
    # 所有不为1的标签的召回率（即仅考虑真实标签为上涨或下跌样本是否被正确分类）
    index = y != 1
    print("上涨下跌召回率：", sum(y_hat[index]==y[index])/sum((index)+1e-6))
    # 所有不为1的标签的准确率（即仅考虑预测为上涨或下跌样本是否是正确）
    index = y_hat != 1
    print("上涨下跌准确率：", sum(y_hat[index]==y[index])/sum((index)+1e-6))
```

```
# 训练完了之后，看看在训练集的准确率（拟合的怎么样）
y_hat = model.predict(train_data)
y = train_label
check_metric(y, y_hat)
```

预测正确的标签数： 13223
总体正确率： 0.66115
真实标签为0样本的正确预测个数： 0
真实标签为1样本的正确预测个数： 13223
真实标签为2样本的正确预测个数： 0
上涨下跌召回率： 0.0
上涨下跌准确率： 0.0

```
# 再看看测试集的准确率
y_hat = model.predict(test_data)
y = test_label
check_metric(y, y_hat)
```

预测正确的标签数： 2887
总体正确率： 0.7195912263210369
真实标签为0样本的正确预测个数： 0
真实标签为1样本的正确预测个数： 2887
真实标签为2样本的正确预测个数： 0
上涨下跌召回率： 0.0
上涨下跌准确率： 0.0

Step3：模型优化

- 可能的问题1：类别不均匀，静止tick太多
- 可能的解决办法：
 - 根据样本频率做加权

```
## 对样本依据类别加权:  
model2 = SVC(class_weight='balanced')  
model2.fit(train_data,train_label)
```

```
SVC  
SVC(class_weight='balanced')
```

- 结果：总体准确率下降了，但是只考虑上涨/下跌的情况，召回率和准确率都有大幅提升

```
print("总体正确率:", sum(y_hat == y)/len(y_hat))  
  
## 我们更关心上涨下跌情况的预测  
# 所有不为1的标签的召回率 (即仅考虑真实标签为上涨或下跌样本是否被正确分类)  
index = y != 1  
print("训练集上涨下跌召回率:", sum(y_hat[index]==y[index])/sum((index)+1e-6))  
# 所有不为1的标签的准确率 (即仅考虑预测为上涨或下跌样本是否是正确)  
index = y_hat != 1  
print("训练集上涨下跌准确率:", sum(y_hat[index]==y[index])/sum((index)+1e-6))
```

总体正确率: 0.5348
训练集上涨下跌召回率: 0.36402430566818034
训练集上涨下跌准确率: 0.271187707622863

对上涨和下跌的召回率和准确率都大幅上升!

Step3 : 模型优化

- 可能的问题1: 类别不均匀, 静止tick太多
- 可能的问题2: 样本稀疏
- 可能的问题3: 存在噪音 (误分类点与惩罚系数)
- 可能的问题4:

自行继续探索.....

Step3 : 模型优化

■ 如何进行参数的排列组合搜索？

■ GridSearch: 网格搜索

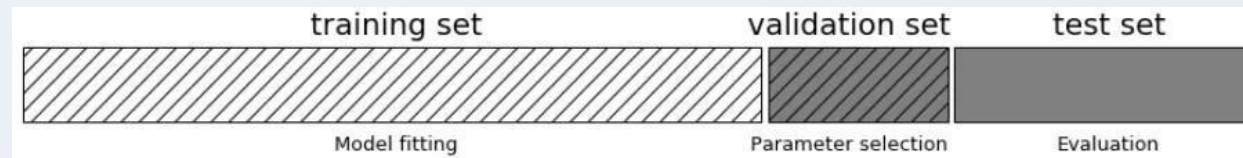
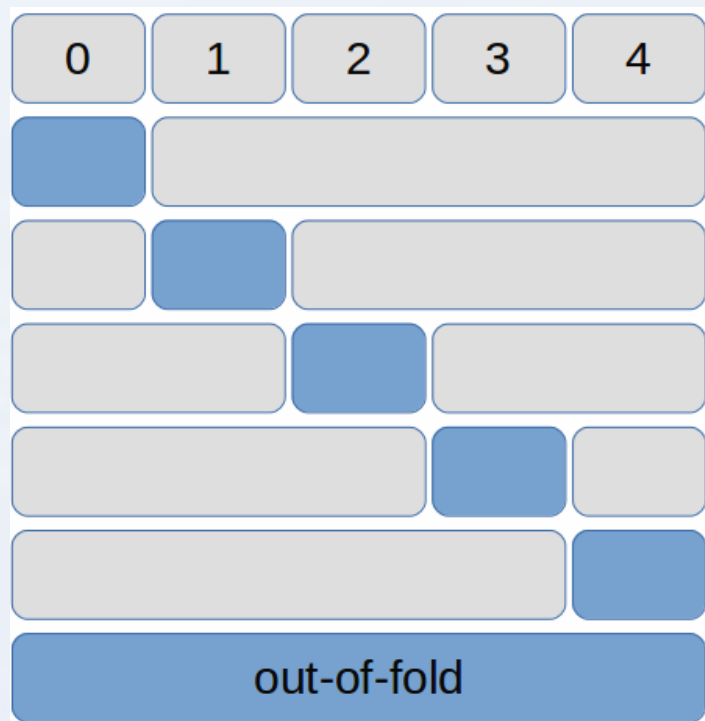
	C = 0.001	C = 0.01	...	C = 10
gamma=0.001	SVC(C=0.001, gamma=0.001)	SVC(C=0.01, gamma=0.001)	...	SVC(C=10, gamma=0.001)
gamma=0.01	SVC(C=0.001, gamma=0.01)	SVC(C=0.01, gamma=0.01)	...	SVC(C=10, gamma=0.01)
...	...	...	...	...
gamma=100	SVC(C=0.001, gamma=100)	SVC(C=0.01, gamma=100)	...	SVC(C=10, gamma=100)

Step3 : 模型优化

■ 如何进行参数的排列组合搜索？

■ GridSearch: 网格搜索

■ CrossValidation: 交叉验证 (股票时间序列?)



Step3 : 模型优化

■ 如何进行参数的排列组合搜索？

- GridSearch: 网格搜索
- CrossValidation: 交叉验证
- sklearn.model_selection.GridSearchCV

```
print("总体正确率: ", sum(y_hat == y)/len(y_hat))

## 我们更关心上涨下跌情况的预测
# 所有不为1的标签的召回率 (即仅考虑真实标签为上涨或下跌样本是否被正确分类)
index = y != 1
print("训练集上涨下跌召回率: ", sum(y_hat[index]==y[index])/sum((index)+1e-6))
# 所有不为1的标签的准确率 (即仅考虑预测为上涨或下跌样本是否是正确)
index = y_hat != 1
print("训练集上涨下跌准确率: ", sum(y_hat[index]==y[index])/sum((index)+1e-6))
```

总体正确率: 0.5348
训练集上涨下跌召回率: 0.36402430566818034
训练集上涨下跌准确率: 0.271187707622863

对上涨和下跌的召回率和准确率都大幅上升!

类别平衡后的结果

```
# == 训练集指标 ==
y_hat = Grid.best_estimator_.predict(train_data)
y = train_label

print("总体正确率: ", sum(y_hat == y)/len(y_hat))

## 我们更关心上涨下跌情况的预测
# 所有不为1的标签的召回率 (即仅考虑真实标签为上涨或下跌样本是否被正确分类)
index = y != 1
print("训练集上涨下跌召回率: ", sum(y_hat[index]==y[index])/sum((index)+1e-6))
# 所有不为1的标签的准确率 (即仅考虑预测为上涨或下跌样本是否是正确)
index = y_hat != 1
print("训练集上涨下跌准确率: ", sum(y_hat[index]==y[index])/sum((index)+1e-6))
```

总体正确率: 0.16875
训练集上涨下跌召回率: 0.4980064984313371
训练集上涨下跌准确率: 0.16874983125012438

为什么效果并没有特别显著的提升?

- scoring的方式?
- 参数覆盖范围太小?

数据分析就是一步一步思考, 一点一点接近“更优解”的探索过程

问题2: SVM参数这么多, 有没有什么简单的参数调整方法?

```
model = SVC()
grid_params = [{'kernel': ['rbf', 'linear'], 'C': [0.5, 1, 5], 'class_weight': ['balanced']}]
Grid = GridSearchCV(model, grid_params, cv = 5, scoring = 'accuracy', refit=True, n_jobs = 6)
Grid.fit(train_data, train_label)
```

GridSearchCV

```
GridSearchCV(cv=5, estimator=SVC(), n_jobs=6,
             param_grid=[{'C': [0.5, 1, 5], 'class_weight': ['balanced'],
                           'kernel': ['rbf', 'linear']}],
             scoring='accuracy')
```

best_estimator_: SVC

```
SVC(C=0.5, class_weight='balanced', kernel='linear')
```

SVC

```
SVC(C=0.5, class_weight='balanced', kernel='linear')
```

Step 4: 特征工程 (Featuer Engeering)

- 如何把先验信息加入模型中?
 - 特征(自变量)
- 增加有效信息:
 - 根据领域知识加入更有效的特征/特征组合
- 去除无效干扰:
 - 去除数据噪音:
 - 降维: PCA
 - Normalization: Z-score

Step 4: 特征工程 (Feature Engineering)

■ 做一些简单处理

网上查找、研报、论文、AI...

- 已做了量价标准化处理：价格相对昨收的涨跌、订单量转化为换手率%
- 为简单模型提供更多特征, 复杂的模型可能自己就能抽取出来这些

价格+1 (从涨跌幅还原到对前收盘价的比例)

```
new_df['bid1'] = new_df['n_bid1'] + 1
new_df['bid2'] = new_df['n_bid2'] + 1
new_df['bid3'] = new_df['n_bid3'] + 1
new_df['bid4'] = new_df['n_bid4'] + 1
new_df['bid5'] = new_df['n_bid5'] + 1
new_df['ask1'] = new_df['n_ask1'] + 1
new_df['ask2'] = new_df['n_ask2'] + 1
new_df['ask3'] = new_df['n_ask3'] + 1
new_df['ask4'] = new_df['n_ask4'] + 1
new_df['ask5'] = new_df['n_ask5'] + 1
```

对量取对数

```
new_df['bsize1'] = new_df['n_bsize1'].map(np.log)
new_df['bsize2'] = new_df['n_bsize2'].map(np.log)
new_df['bsize3'] = new_df['n_bsize3'].map(np.log)
new_df['bsize4'] = new_df['n_bsize4'].map(np.log)
new_df['bsize5'] = new_df['n_bsize5'].map(np.log)
new_df['asize1'] = new_df['n_asize1'].map(np.log)
new_df['asize2'] = new_df['n_asize2'].map(np.log)
new_df['asize3'] = new_df['n_asize3'].map(np.log)
new_df['asize4'] = new_df['n_asize4'].map(np.log)
new_df['asize5'] = new_df['n_asize5'].map(np.log)
new_df['amount'] = new_df['amount_delta'].map(np.log1p)
```

量价组合

```
new_df['spread1'] = new_df['ask1'] - new_df['bid1']
new_df['spread2'] = new_df['ask2'] - new_df['bid2']
new_df['spread3'] = new_df['ask3'] - new_df['bid3']
new_df['mid_price1'] = new_df['ask1'] + new_df['bid1']
new_df['mid_price2'] = new_df['ask2'] + new_df['bid2']
new_df['mid_price3'] = new_df['ask3'] + new_df['bid3']
new_df['weighted_ab1'] = (new_df['ask1'] * new_df['n_bsize1'] + new_df['bid1'] * new_df['n_asize1']) / (new_df['ask1'] + new_df['bid1'])
new_df['weighted_ab2'] = (new_df['ask2'] * new_df['n_bsize2'] + new_df['bid2'] * new_df['n_asize2']) / (new_df['ask2'] + new_df['bid2'])
new_df['weighted_ab3'] = (new_df['ask3'] * new_df['n_bsize3'] + new_df['bid3'] * new_df['n_asize3']) / (new_df['ask3'] + new_df['bid3'])

new_df['relative_spread1'] = new_df['spread1'] / new_df['mid_price1']
new_df['relative_spread2'] = new_df['spread2'] / new_df['mid_price2']
new_df['relative_spread3'] = new_df['spread3'] / new_df['mid_price3']
```

均线特征

```
new_df['ask1_ma5'] = new_df['ask1'].rolling(window=5, min_periods=1).mean()
new_df['ask1_ma10'] = new_df['ask1'].rolling(window=10, min_periods=1).mean()
new_df['ask1_ma20'] = new_df['ask1'].rolling(window=20, min_periods=1).mean()
new_df['ask1_ma40'] = new_df['ask1'].rolling(window=40, min_periods=1).mean()
new_df['ask1_ma60'] = new_df['ask1'].rolling(window=60, min_periods=1).mean()
new_df['bid1_ma5'] = new_df['bid1'].rolling(window=5, min_periods=1).mean()
new_df['bid1_ma10'] = new_df['bid1'].rolling(window=10, min_periods=1).mean()
new_df['bid1_ma20'] = new_df['bid1'].rolling(window=20, min_periods=1).mean()
new_df['bid1_ma40'] = new_df['bid1'].rolling(window=40, min_periods=1).mean()
new_df['bid1_ma60'] = new_df['bid1'].rolling(window=60, min_periods=1).mean()
```

这样写并不高效
此处只是为了方便展示

Step 4: 特征工程 (Featuer Engeering)

■ 再次训练，效果大幅提升

```
In [10]: 1 %%time
          2 ## 对样本依据类别加权:
          3 model = SVC(class_weight='balanced')
          4 model.fit(train_data, train_label)
```

CPU times: user 2min 10s, sys: 395 ms, total: 2min 10s
Wall time: 2min 10s

Out[10]: SVC(class_weight='balanced')

```
In [11]: 1 # 训练集
          2 y_hat = model.predict(train_data)
          3 y = train_label
          4 # 总体准确率:
          5 print("总体准确率: ", sum(y_hat == y)/len(y_hat))
          6 # 所有不为1的标签的召回率 (即仅考虑真实标签为上涨或下跌样本是否被正确分类)
          7 index = y != 1
          8 print("训练集上涨下跌召回率: ", sum(y_hat[index]==y[index])/sum(index))
          9 # 所有不为1的标签的准确率 (即仅考虑预测为上涨或下跌样本是否是正确)
          10 index = y_hat != 1
          11 print("训练集上涨下跌准确率: ", sum(y_hat[index]==y[index])/sum(index))
```

总体准确率: 0.5074

训练集上涨下跌召回率: 0.568785523721146

训练集上涨下跌准确率: 0.3862544308782985

简单回顾

- 我们做了什么操作？
 - 观察数据
 - 清洗和整理输入数据（/标注）
 - 选择一个合适的模型进行训练
 - 根据训练结果和数据本身的特性进行参数优化

简单回顾

■ 我们还可以做什么？

- 更换其他的模型：决策树，随机森林，Adaboost等等

```
1 %pylab inline
2 import pandas as pd
3 import numpy as np
4 import time
5 from sklearn.grid_search import GridSearchCV
6 from sklearn.ensemble import (RandomForestClassifier, ExtraTreesClassifier, AdaBoostClassifier, \
7                               GradientBoostingClassifier)
8 from sklearn.tree import DecisionTreeClassifier
9 from sklearn.svm import SVC
10 from sklearn import metrics
11 from sklearn.linear_model import LogisticRegression
```

简单回顾

■ 我们还可以做什么？

- 更换其他的模型：决策树，随机森林，Adaboost等等

- 模型集成：

 - 用多个模型进行投票，选择最终结果

 - Boosting/Bagging

- 更精细的处理输入数据

 - 合适的标准化：min-max；z-score；

 - 维度增加/减少，分析不同特征的重要性

THANKS!